Informe técnico — TP4 a TP8

Food Store — Base de Datos II Franco Rios y Santino Barone — Noviembre de 2026

Este informe resume los trabajos que van del TP4 al TP8, incluyendo las entregas de seguridad y respaldo de las unidades 5 y 6. Los nombres de las carpetas del repositorio no siempre coinciden con el número de unidad, así que agrupamos por trabajo entregado para no dejar temas afuera. No repetimos todas las consignas ni cada comando: contamos qué implementamos, cómo lo probamos, qué resultados obtuvimos y por qué tomamos determinadas decisiones. Al final incluimos el uso que le dimos a la IA.

1. Qué implementamos en cada trabajo

TP4 — consultas analíticas y optimización. Partimos de dos consultas lentas sobre la base masiva del TP3: facturación por producto y ranking de clientes por gasto. Las medimos con `EXPLAIN (ANALYZE, BUFFERS)`, leímos los planes y probamos dos caminos: agregar índices y reescribir con CTE para pre-agregar antes de unir. También analizamos críticamente los planes de join que propuso la IA.

TP5 — índices y vistas. Diseñamos índices B-Tree y cubrientes para tres consultas típicas (pedidos por cliente y fecha, catálogo por precio y agregación de detalle), y creamos vistas de negocio, una vista con criterio de seguridad y una vista materializada para el reporte mensual. Verificamos que las vistas devolvieran lo mismo que la consulta manual y que ocultaran las columnas privadas de `cliente`.

TP6 — FNBC y desnormalización. Analizamos la relación `control_lote_almacen` a partir de dos reglas de negocio, calculamos claves candidatas y encontramos que estaba en 3FN pero no en FNBC por `ResponsableControlID → DepositoID`. La descompusimos sin pérdida en `responsable_control` y `control_lote`. Después implementamos una tabla de resumen `resumen_venta_categoria_dia` mantenida por disparadores para reemplazar una consulta que recorría cientos de miles de detalles.

Unidad 5 — roles, auditoría y protección de datos. Armamos una capa de seguridad sobre el esquema comercial: tablas `usuario` e `intento_login`, función `fn_autenticar`, roles de grupo y de login, `GRANT`/`REVOKE` por tabla y por columna, auditoría nativa, un simulacro de incidente y una tabla `usuario_anon` para reportes. La idea fue que cada rol pudiera hacer solo lo suyo y que ningún rol de aplicación pudiera administrar cuentas.

Unidad 6 — respaldo, monitoreo y mantenimiento. Escribimos la especificación y cuatro piezas: `backup_foodstore.ps1`, `health_check.sql`, `maintenance.sql` y `restore_test.ps1`. El respaldo se hace en formato custom; el health check corre ocho controles; el mantenimiento usa `VACUUM (ANALYZE)` y ajusta autovacuum; la prueba de restauración crea una base aislada y valida conteos e integridad.

TP7 — modelo documental. Migramos cinco pedidos representativos a MongoDB como documentos con `lineas_pedido` embebidas. Conservamos el `_id` de origen, el estado, la forma de pago y los precios históricos. Después comparamos las mismas consultas en PostgreSQL y en MongoDB para ver qué se gana y qué se pierde con cada modelo.

TP8 — la tablet sin señal. Diseñamos una base SQLite `STRICT` con `producto_cache`, `pedido_borrador`, `item_borrador` y `cola_sync`, más las tablas de instantánea. Exportamos el catálogo desde PostgreSQL a un CSV, lo importamos en la tablet y escribimos un semáforo que clasifica cada solicitud como `RECHAZADO`, `CONFLICTO` o `SIMULADO_APROBADO`. La tablet nunca confirma ventas ni modifica stock o precios del servidor.

2. Cómo probamos su funcionamiento

En el **TP4** ejecutamos las consultas antes y después de cada cambio, siempre sobre la misma base y con varias corridas para no confundir caché con mejora. Los ajustes de `work_mem` se hicieron con `SET LOCAL` dentro de una transacción, así quedaban solo para esa prueba. Además de medir tiempos, comparamos que las variantes devolvieran exactamente las mismas filas usando `EXCEPT` en los dos sentidos.

En el **TP5** medimos cada índice con `EXPLAIN ANALYZE` antes y después, y comparamos la vista materializada contra la consulta original. Para las vistas normales hicimos una consulta de control equivalente y verificamos que

devolviera la misma cantidad de filas y los mismos datos; también comprobamos que `vista_pedidos_cliente` no expusiera `email` ni `telefono`.

En el **TP6** la validación de FNBC fue formal: clausuras, claves candidatas y verificación de cada dependencia. La migración se hizo dentro de una transacción y se comprobó con `EXCEPT` en ambos sentidos y con un bloque `DO` que aborta si la vista de compatibilidad no reconstruye la relación original. Para la desnormalización medimos antes y después con tres corridas, ejecutamos nueve mutaciones dentro de una transacción y corrimos una auditoría con `FULL OUTER JOIN` para detectar filas faltantes, sobrantes o totales distintos. El `rollback` del resumen está probado y no toca las tablas fuente.

En la **Unidad 5** probamos los permisos con un script que intenta operaciones permitidas y prohibidas para cada rol. La prueba de auditoría se ejecutó como `app_foodstore` e incluyó lectura, inserción, actualización y un login fallido, todo cerrado con `ROLLBACK` para no dejar datos. El simulacro de incidente se corrió como `reportes_foodstore`: seis intentos de autenticación fallidos, uno exitoso, lectura de las 6.000 cuentas en las columnas permitidas y tres intentos de escalamiento (leer `password_hash`, hacer `SET ROLE admin_datos` y otorgarse el rol). Los tres fueron rechazados. También comparamos los agregados de `usuario` contra `usuario_anon` para verificar que la anonimización no cambiara los totales.

En la **Unidad 6** hicimos un respaldo real y una prueba negativa contra una base inexistente para confirmar que el script devolviera código 1 y borrara el archivo parcial. El simulacro de desastre consistió en restaurar ese respaldo en `foodstore_restore_test`, una base separada, y validar los conteos contra el manifiesto previo, las claves foráneas, los `CHECK` y los totales. El health check se corrió antes y después del mantenimiento.

En el **TP7** migramos los cinco pedidos y comparamos cantidad de documentos, `_id` y resultados de las consultas equivalentes. También contrastamos dos afirmaciones de la IA: que MongoDB evita los JOIN y que se pierde integridad referencial. La primera se confirmó; la segunda la relativizamos, porque el motor no tiene claves foráneas pero la aplicación o un esquema JSON pueden validar las referencias.

En el **TP8** trabajamos sobre una copia de la base con datos inventados y probamos C2 a C6 y los casos de restricción C7a y C7b. C2 sube el precio de la leche, da `CONFLICTO` y luego, cuando el cliente acepta, genera una solicitud nueva con otro UUID y `SIMULADO_APROBADO`. C3 pide más manzanas de las que hay y da `RECHAZADO` por stock. C4 da de baja lógica un producto: el catálogo deja de mostrarlo, pero el ítem local se conserva para poder explicar el rechazo. C5 no tiene cambios y queda en verde. C6 reenvía el mismo UUID y la clave primaria lo rechaza. C7a y C7b comprueban que una cantidad cero y un producto inexistente fallan sin dejar filas. Después corrimos otra vez el semáforo para confirmar que no vuelve a evaluar lo ya resuelto y cerramos con la verificación en PostgreSQL.

3. Qué resultados obtuvimos

Trabajo	Resultado principal
TP4	Facturación por producto: 1,18× más rápida con índice cubriente y CTE. Ranking de clientes: 1,40× con índice parcial y pre-agregación. En la competencia interna bajamos de 679,43 ms a 417,91 ms (1,63×).
TP5	Los índices redujeron las consultas de 0,115 a 0,065 ms y de 0,210 a 0,059 ms; la vista materializada pasó de 726,968 a 17,015 ms. Las vistas devolvieron los mismos datos que las consultas de control y ocultaron las columnas privadas.
TP6	La descomposición eliminó la violación de FNBC y la reunión devolvió las tres filas originales, con diferencia simétrica 0. El resumen bajó la consulta de 244,707 a 0,183 ms en la base canónica (unas 1.337×) y de 289,556 a 0,239 ms en el laboratorio (1.212×). La auditoría quedó en 0 filas y las nueve mutaciones se sincronizaron.
Unidad 5	El esquema de permisos bloqueó los tres intentos de escalamiento. El `password_hash` no fue legible. El impacto real fue la lectura de metadatos y datos de contacto de 6.000 cuentas, que quedó documentado en el informe post-incidente.
Unidad 6	Respaldo de 9,82 MB en 1,9 s. El health check inicial detectó 46,34 % de tuplas muertas en `pedido` (172.684); después del `VACUUM` quedó 8/8 OK. Restauración completa en 5,69 s (RTO) con un RPO observado de 0,02 h y 12/12 validaciones de conteos e integridad.

TP7	Los cinco documentos migrados conservaron `_id` y precios históricos. Las consultas comparadas coincidieron: 2/2 pedidos pagados y 3/3 pedidos con el producto 1.
TP8	Instantánea de 16 productos, 13 visibles en el catálogo. La simulación inicial dejó 4 verdes, 1 amarillo y 5 rojos. La verificación final en PostgreSQL dio 3 pedidos, 4 detalles y stock 295: `VERDE: pedidos y stock sin cambios`.

El informe post-incidente de la Unidad 5 fue importante porque nos obligó a separar lo que el log realmente prueba de lo que la IA suponía. La IA decía que no había habido lectura sensible, pero el log mostraba la lectura masiva de emails y nombres; y afirmaba que no hubo modificaciones cuando `fn_autenticar` sí escribe en `intento_login` y actualiza `ultimo_acceso`. Esa contradicción quedó documentada y sirvió para ajustar la contención y las recomendaciones.

4. Qué decisiones de diseño tomamos

En el **TP4** preferimos un índice cubriente y uno parcial antes que llenar la tabla de índices. El cubriente evita volver a la tabla para `cantidad` y `precio_unitario`; el parcial de `pedido` sirve solo para los estados que se consultan. La reescritura con CTE baja el volumen antes del join, y el `work_mem` se ajusta por sesión y no de forma global. No forzamos planes con `enable_*`: medimos y dejamos que el optimizador elija.

En el **TP5** elegimos índices compuestos y cubrientes según el filtro real, y una vista materializada para el reporte mensual porque se consulta mucho más de lo que se actualiza. El refresco se programa una vez al día: el usuario ve una foto con algunas horas de antigüedad, pero no se recalcula toda la historia en cada consulta. La vista de seguridad expone solo las columnas necesarias.

En el **TP6** aplicamos el algoritmo de descomposición binaria sobre la dependencia violatoria y verificamos la reunión sin pérdida. Documentamos una limitación: la descomposición no conserva como restricción local la unicidad original de `(LoteID, DepositoID)`. Para la consulta del top 5 elegimos una tabla de resumen por disparadores en lugar de una vista materializada, porque el `REFRESH` tendría que recorrer otra vez cientos de miles de filas; el resumen se actualiza por delta en la misma transacción y es reversible.

En la **Unidad 5** aplicamos mínimo privilegio de verdad: roles de login sin permisos directos, permisos por columna, `REVOKE` global sobre `public` y `fn_autenticar` como `SECURITY DEFINER` con `search_path` fijo. Descartamos crear un rol de aplicación con `CREATEROLE` y `ADMIN OPTION`, porque permitía reotorgarse privilegios y no daba contención real. La administración de roles queda en el DBA. También separamos `usuario_anon` para que los reportes no necesiten datos personales.

En la **Unidad 6** elegimos respaldo custom, restauración en una base aislada y health check que falla con excepción si algún control queda en `FAIL`. No usamos `VACUUM FULL` como tarea periódica porque bloquea la tabla; preferimos `VACUUM (ANALYZE)` y bajar los umbrales de autovacuum en `detalle_pedido` (0,05 y 0,02). Los scripts devuelven códigos de salida claros para poder programarlos y alertar.

En el **TP7** decidimos embeber las líneas del pedido. El patrón de acceso es leer el pedido completo, y el precio copiado dentro de la línea conserva la factura histórica aunque cambie el catálogo. No normalizamos `lineas_pedido` en una colección aparte: perderíamos esa foto y volveríamos a necesitar `$lookup`. La conclusión fue usar persistencia políglota: PostgreSQL para la transacción y el stock, MongoDB para la lectura rápida del pedido.

En el **TP8** elegimos instantánea completa en vez de incremental porque la feria no tiene señal y un CSV chico es fácil de reemplazar y auditar. El dinero viaja en centavos enteros para evitar redondeos. Las tablas son `STRICT`, los estados están cerrados y la PK UUID de `cola_sync` da idempotencia en la tablet. Esa protección no alcanza sola: en un sistema real el servidor necesita además una clave única de idempotencia y confirmar la venta en una transacción. Tampoco borramos los ítems locales cuando un producto se da de baja: se conservan para explicar el rechazo.

5. Qué herramientas de IA usamos

La herramienta principal fue **OpenCode** (modelo DeepSeek V4.1 Flash Free), con `psql`, `pg_dump`, PowerShell y SQLite para ejecutar y verificar. En el TP5 usamos **Kiro** para las especificaciones y en la Unidad 5 lo reemplazamos

por OpenCode, tal como quedó aclarado en el README. La IA se usó para proponer esquemas, reescribir consultas, redactar scripts y documentos, y hacer un code review de todos los TP.

Algunos prompts concretos fueron: “Necesito cuatro tablas SQLite STRICT para una tablet sin señal; el servidor decide precio y stock y la tablet solo arma borradores”; “escribí un semáforo que compare las solicitudes PENDIENTE contra la instantánea y no re-evalúe las resueltas”; “revisá todos los scripts de TP1 a TP8 y proponé hallazgos con decisión, motivo y forma de comprobarlos, con al menos uno descartado”; y, para el TP7, un prompt de arquitecto para comparar PostgreSQL y MongoDB en el almacenamiento de pedidos.

Aceptamos la estructura general del esquema SQLite, la precedencia del semáforo, el índice cubriente, la vista materializada, el resumen por disparadores, los roles con permisos por columna y el respaldo con prueba de restauración. **Corregimos** varias cosas: el precio pasó de `REAL` a centavos enteros; agregamos `snapshot_raw` porque los booleanos `t/f` rompían una tabla `STRICT`; unificamos el motivo de stock cero; corregimos el manejo de `stderr` en PowerShell y una ambigüedad de columna en el health check.

Descartamos propuestas con fundamento: usar `VACUUM FULL` en forma periódica (bloquea ventas), reemplazar la excepción del trigger de resumen por un upsert silencioso (ocultaría una desincronización que la auditoría debe ver), normalizar `lineas_pedido` en MongoDB (perdería el precio histórico y la lectura directa), borrar los ítems locales en C4 (rompe la FK y borra la evidencia), reutilizar el UUID en C2 (rompe la idempotencia y el historial), crear un rol de aplicación con `CREATEROLE` (no contiene privilegios) y dejar `log_statement = 'all'` o `trust` en producción (son decisiones de laboratorio). Todo esto está en las bitácoras y en las DUIA de cada trabajo.

Cierre

Los trabajos nos dejaron una idea común: primero se entiende el problema y después se elige la herramienta. En el TP4 y el TP5 el problema era el acceso a los datos; en el TP6, la redundancia y el costo de recalcular; en la Unidad 5, quién puede hacer qué; en la Unidad 6, cómo recuperarnos si algo falla; en el TP7, qué modelo conviene para cada consulta; y en el TP8, cómo trabajar sin conexión sin romper la verdad del servidor. En todos los casos probamos, medimos y dejamos evidencia, y descartamos las sugerencias de IA que no resistieron esa prueba.